

Auth Server Architecture

Sep 24, 2026 · @Simone

Overview

This implements the server side of App Attest registration and session authentication for SecureChat — the piece that Apple's `DCAppAttestService` on the device talks to. It lives at `AuthServer/` in the repo, as its own Gradle project, separate from the iOS codebase.

Four endpoints, matching the contract already specified in `Architecture/architecture-decisions.md` §6 and `Architecture/attestation-assertion-workflow.md`:

Endpoint	Purpose
GET <code>/challenge</code>	Issues a one-time registration challenge, bound to the caller's identity public key
POST <code>/register</code>	Verifies a device's App Attest attestation and mints (or returns) an account UUID
GET <code>/session/nonce</code>	Issues a one-time, short-lived nonce for authenticating a request
POST <code>/session</code>	Verifies an App Attest assertion against a stored account and issues a session token

It sits between two things that already existed before this work: **AppAttestKit** (the Swift module on the device that talks to the Secure Enclave via `DCAppAttestService`) and **the eventual SecureChat app**, which will call these endpoints instead of the `LocalFakeTransport` mock currently wired into the `AppAttestTestApp` test harness. This server doesn't yet talk to a real device — everything so far has been verified with synthetic requests (`curl`) against the live containers, described in the status section near the end.

The rest of this doc walks through *why* it's built the way it is (language choice, a real protocol trade-off), *how* each piece works (the four endpoints, Redis, Postgres), *how* it runs (Docker), and *what's actually been verified* versus what's still a documented gap.

Why Kotlin + Ktor + devicecheck-appattest

The project's own architecture docs flag "server language/stack" as an explicitly open question, specifically because App Attest verification quality varies a lot between ecosystems — and because the docs already rule out hand-rolling ASN.1/CBOR parsing ourselves ("a well-known CVE farm"). So the language choice came down to: which ecosystem has a genuinely trustworthy, maintained library for this?

I checked three ecosystems before picking anything:

- **Go:** fragmented. At least six independent small projects (`splitsecure/go-app-attest` , `predicat-inc/go-app-attest` , `takimoto3/app-attest` , `iam047801/appattest` , `bas-d/appattest` , `kacy/device-attestation`), no clear leader, low visibility on any of them.
- **Node/TypeScript:** same pattern — `uebelack/node-app-attest` , `srinivas1729/appattest-checker-node` , `figamore/app-attest-server` , `Staberman/app-attest-gate` , `bradford-tech/supabase-integrity-attest` . Several looked very recently started.
- **Kotlin:** one clear standout, [veehaitech/devicecheck-appattest](#) . Published to Maven Central, built on Bouncy Castle (a real, audited crypto library) for the X.509/ASN.1 work and Jackson for CBOR — not hand-rolled parsing. It has a real test suite with captured fixtures across iOS versions, and it implements Apple's own "Validating Apps That Connect to Your Server" spec directly.

The honest caveat: it's at version `0.9.4` — pre-1.0, single maintainer (Vincent Hauptert). That's a real risk, mitigated by pinning the exact version in `build.gradle.kts` rather than auto-upgrading, and forking it if it ever goes stale. That risk still seemed smaller than reimplementing X.509 chain validation and CBOR parsing ourselves.

For the web framework: **Ktor**, not Spring Boot. The auth server's whole job is "nearly free, stateless, horizontally trivial" (architecture-decisions.md §6) — four endpoints, no views, no heavy domain model. Ktor gives you that directly; Spring's DI/autoconfiguration machinery would be pure overhead for a service this shape.

The clientDataHash trade-off

This is the one real protocol change that came out of building this server — worth understanding because it touches security reasoning, not just code shape.

The original design (from `account-keys-reference.md` , decided earlier in the project): `clientDataHash = SHA256(challenge || identityPublicKey)` . The App Attest key signs over a hash that includes the identity public key, so the server can conclude "this identity

key came from a genuine app on real hardware" — a binding that's provable from the signed attestation object alone, independent of whether the network transport for `/register` can be trusted.

The problem: `devicecheck-appattest`'s `AttestationValidator.validate()` computes `clientDataHash` internally as `SHA256(serverChallenge)` — plain, no identity binding — and that's buried in a `private` method with no override seam. There's no way to hand it a different hash formula. And since none of the library's internal verification steps (chain validation, nonce check, `rpIdHash` check, counter check) are separately callable, there's no way to reuse *most* of the library while only swapping the nonce computation — it's all-or-nothing.

Two options, one call: fork ~150 lines of the library's most security-critical code to change one hash formula, or change the protocol to match what the library expects and recover the identity binding a different way. I flagged this explicitly rather than picking silently, since it's a real security trade-off. The choice: match the library.

What changed: `clientDataHash = SHA256(challenge)` now, no identity binding inside the hash. Instead, `/challenge` takes `identityPublicKey` as a required input and stores `challenge → identityPublicKey` in Redis at issuance (single-use, same TTL as the challenge). `/register` then checks that the submitted `identityPublicKey` matches what was stored when the challenge was issued, before ever calling into the verification library. Functionally the same anti-substitution guarantee — an attacker can't register someone else's attestation under a different identity key — but it now depends on TLS integrity for `/challenge` and `/register` specifically, rather than being provable from the signed attestation object alone. That's a real, if modest, reduction in defense-in-depth, accepted in exchange for keeping 100% of the ASN.1/CBOR/X.509 verification logic inside the maintained library, completely untouched.

This is documented as a revision (not silently) in `account-keys-reference.md`, `attestation-assertion-workflow.md`, and `architecture-decisions.md`'s decision log, plus a doc-comment update in `AttestationCoordinator.swift` — nothing in shipped Swift code actually implemented the old formula yet, so nothing had to be un-written.

Request flow walkthroughs

All four routes live in `Routes.kt`. Base64 throughout is URL-safe, unpadding.

GET `/challenge?identityPublicKey=<base64>`

1. Reject with `400 identity_public_key_required` if the query param is missing.

2. `RegistrationChallengeStore.issue()` generates 32 random bytes, stores `challenge:register:<challenge> → identityPublicKey` in Redis with a 15-minute TTL, and returns the raw challenge to the caller.

`POST /register` — body: { `keyId`, `challenge`, `attestation`, `identityPublicKey` }

1. **Idempotency check first:** look up an existing account by `keyId`. If found, return its `account_uuid` immediately — this has to happen *before* touching the challenge (see the bug story below).
2. Consume the challenge from Redis (single-use `GETDEL`). Missing/expired → `400 challenge_invalid_or_expired`.
3. Compare the submitted `identityPublicKey` against the one stored at challenge-issuance time. Mismatch → `400 identity_mismatch`.
4. Hand `attestationObject`, `keyId`, and the raw challenge to `AppAttestVerification.validateAttestation()` — this is where the library does chain validation, nonce check, rpIdHash check, counter-is-zero check, and keyId-matches-pubkey check, all in one call. Any failure (including a CBOR parse failure before the library's own exception types even apply) → `400 attestation_invalid:<ExceptionClassName>`.
5. `AccountRepository.registerOrReturnExisting()` — insert-or-return, keyed on `keyId`, storing the attested EC public key.
6. Respond with the account UUID.

`GET /session/nonce`

Same shape as `/challenge`, but no identity binding — just a random 32 bytes with a 60-second TTL, since nothing is cached against a session nonce the way an attestation is cached against a registration challenge.

`POST /session` — body: { `keyId`, `assertion`, `nonce`, `body` }

1. Consume the nonce (single-use). Missing/expired → `400 nonce_invalid_or_expired`.
2. Look up the account by `keyId`. Not found → `404 unknown_key_id`.
3. Reconstruct `clientData = nonce || body` and hand it, the stored attested public key, and the stored counter to `AppAttestVerification.validateAssertion()`. The library hashes `clientData` itself internally — that's the one place the naming lines up cleanly with what the device already computed (`payloadHash = SHA256(nonce + body)` fed into `generateAssertion`).

4. On success, the library returns the assertion's parsed counter (`signCount`).

`AccountRepository.advanceCounter()` does a compare-and-swap `UPDATE ... WHERE counter = ?` — if another concurrent request already advanced it, this fails and the request is rejected with `401 counter_conflict` rather than silently accepting a stale transition.

5. Issue an opaque session token via `SessionTokenStore`, respond with it plus the account UUID.

Redis — ephemeral state

Everything in `RedisStore.kt` is short-lived by design, using `JedisPooled` (Jedis 5.2, a plain synchronous client — no need for anything reactive at this request volume). Three small classes, each wrapping one concern:

Store	Key prefix	TTL	Single-use?
<code>RegistrationChallengeStore</code>	<code>challenge:register:</code>	15 min	Yes — <code>GETDEL</code> on consume
<code>SessionNonceStore</code>	<code>challenge:session:</code>	60 s	Yes — <code>GETDEL</code> on consume
<code>SessionTokenStore</code>	<code>session:token:</code>	15 min	No — plain <code>GET</code> , re-checked on every session-scoped call

The 15-minute vs. 60-second TTL split matters and was actually a real bug in the docs before this work: `architecture-decisions.md` said 60 seconds for the registration challenge, while `attestation-assertion-workflow.md` separately (and correctly) explained *why* it needs to be longer — a cached attestation is bound to its challenge and the key can only ever be attested once, so if the app crashes between attesting and submitting, it needs a challenge that's still alive when it resumes and retries the upload. I fixed the inconsistency in `architecture-decisions.md` before writing any server code, so the two docs agree now.

Why an opaque token instead of a JWT for `SessionTokenStore`: a self-contained signed JWT can't be revoked without a blocklist, and a Redis round-trip already happens on every session-scoped call anyway (to check the token), so a JWT's main advantage — avoiding a lookup — doesn't apply here. An opaque token backed by a key you can just delete is simpler and strictly more capable for this use case.

Postgres — durable state

One table, plain JDBC via HikariCP — no ORM. `AccountRepository.migrate()` runs a `CREATE TABLE IF NOT EXISTS` on startup, which is fine for a single-table schema at this stage but isn't how this should stay once the schema needs to evolve (a real migration tool belongs there eventually, noted as a gap).

```
CREATE TABLE accounts (
    account_uuid  UUID PRIMARY KEY,
    key_id        BYTEA NOT NULL UNIQUE,
    attest_pubkey BYTEA NOT NULL,
    counter       BIGINT NOT NULL DEFAULT 0,
    created_at    TIMESTAMPTZ NOT NULL DEFAULT now()
)
```

`attest_pubkey` stores the raw X.509 `SubjectPublicKeyInfo` DER bytes (`certificate.publicKey.encoded`), reconstructed back into an `ECPublicKey` via `KeyFactory` when needed for assertion verification — no need for a separate encoding scheme, Java's own standard one round-trips fine.

Two concurrency-sensitive operations:

- **registerOrReturnExisting**: `INSERT ... ON CONFLICT (key_id) DO NOTHING`, then read back the row — whether *this* request's insert won or a concurrent one beat it to it, the read-back always returns the row that's actually there. This is what makes `/register` idempotent on `keyId`.
- **advanceCounter**: `UPDATE accounts SET counter = ? WHERE key_id = ? AND counter = ?` — an optimistic-concurrency compare-and-swap. If two requests both read counter `N` and both try to advance it, only one `UPDATE` matches the `WHERE counter = N` clause; the loser gets `0` rows affected and the route rejects with `counter_conflict` rather than silently overwriting.

Docker and docker-compose

The whole point of "local instance first, reliable enough to roll to a real cloud instance" is refusing to let anything be true only on one machine.

Dockerfile is a two-stage build:

```

FROM eclipse-temurin:21-jdk AS build
WORKDIR /workspace
COPY gradlew build.gradle.kts settings.gradle.kts ./
COPY gradle ./gradle
RUN ./gradlew --no-daemon dependencies || true
COPY src ./src
RUN ./gradlew --no-daemon shadowJar

FROM eclipse-temurin:21-jre
WORKDIR /app
COPY --from=build /workspace/build/libs/*-all.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]

```

The build stage never ships — only the final `-all.jar` (the Ktor Gradle plugin's shadow/fat jar, everything bundled including Netty, Jackson, Bouncy Castle, the JDBC driver) makes it into the runtime image, which uses a JRE rather than a full JDK. Copying `build.gradle.kts / gradle/` before `src/` and running `dependencies` as a separate step is a standard layer-caching trick — dependency resolution is the slow part, and it's cached as long as the build file doesn't change, so a source-only edit doesn't re-download the internet.

docker-compose.yml defines three services: `auth-server` (built from that Dockerfile), `postgres:17-alpine`, `redis:7-alpine`. `auth-server` waits on Postgres's healthcheck (`pg_isready`) before starting, not just on the container existing — Postgres accepting TCP connections and Postgres being ready to serve queries aren't the same moment. All the App Attest-specific config (`APPATTEST_TEAM_ID`, `APPATTEST_BUNDLE_ID`, `APPATTEST_ENVIRONMENT`) comes from a gitignored `.env` file via `${VAR}` substitution, never hardcoded into the compose file itself.

The portability part: every environment-specific value is read from an env var in `Config.kt`, with no hardcoded local default that would silently work on one machine and break elsewhere. Locally, those env vars point at the Compose services (`postgres`, `redis` as hostnames, resolved via Compose's built-in DNS). In a real deployment, the *same image* runs with different values — a managed Postgres/Redis connection string, `APPATTEST_ENVIRONMENT=production` once the app ships with the production entitlement. Nothing about moving from local to cloud requires touching the Dockerfile or the application code, only the env vars — which is the whole point of building it this way from day one rather than retrofitting it later.

Error handling, and two real bugs

Errors are structured JSON, `{"error": "machine_readable_code"}`, never raw exception text — handled centrally via Ktor's `StatusPages` plugin in `Application.kt` rather than scattered try/catch in each route. Three tiers: a typed `ApiException` (thrown deliberately by route code via `badRequest()` / `unauthorized()` / `notFound()` helpers) maps to its own status code; Ktor's own `BadRequestException` (thrown by content negotiation when a request body doesn't parse) maps to `400 malformed_request`; and a catch-all `exception<Throwable>` maps anything else to a generic `500 internal_error`, logged server-side but never echoed to the client.

Two real bugs surfaced while actually running the stack and hitting it with `curl` — not just from reading the code:

Bug 1 — idempotency ordering. The first version consumed the single-use registration challenge, *then* checked whether the account already existed. But the documented retry-after-lost-response case (`architecture-decisions.md` §6: "if attestation succeeds but the response is lost, the client retries") means the client resubmits the *same cached challenge* on retry — which would already be consumed from the first attempt, so the retry would see `challenge_invalid_or_expired` instead of the idempotent success the spec requires. Fixed by checking for an existing account *before* touching the challenge at all.

Bug 2 — exception leak. Smoke-testing `/register` with a deliberately garbage (non-CBOR) attestation payload produced a raw `500` with a Jackson parser exception message in the response body, instead of a clean `400`. The cause: malformed CBOR fails to parse *before* the library's own `AttestationException` types are ever thrown, so the original `catch (e: AttestationException)` never caught it. Fixed by widening that catch to `Exception` (the response still only ever exposes the exception's class name, never its message or a stack trace) and adding the `Throwable` catch-all as a second line of defense for anything unanticipated anywhere else in the app.

Current status

Verified, against the live `docker compose` stack (real Postgres, real Redis, not mocks):

- All three containers build and start cleanly; Postgres healthcheck gating works; the schema migration runs without error on startup.
- `GET /challenge` and `GET /session/nonce` round-trip correctly with real Redis TTLs.
- The identity-binding check (`/register` rejecting a submitted `identityPublicKey` that doesn't match the one bound to the challenge at issuance) works as designed.

- Idempotent registration ordering and the exception-leak fix, both re-verified against the rebuilt container after fixing them.
- Malformed request bodies return a clean `400`, not a `500`.

Not yet verified: a real attestation object from an actual device. Everything above used synthetic/garbage payloads via `curl`, since that's all a command line can produce — the CBOR structure and signatures need a real Secure Enclave. The next milestone is replacing `AppAttestTestApp`'s `LocalFakeTransport` mock with a real `AttestationTransport` implementation that calls this server, to get one genuine end-to-end round trip.

Explicitly not built yet (documented in `AuthServer/README.md`, not silently skipped):

- TLS termination — assumed to sit in front of this process (reverse proxy or load balancer), not handled by it directly.
- Rate limiting (`architecture-decisions.md` §9 — Redis token buckets, IP-based before attestation, `keyId`-based after).
- Automated tests for the wiring itself. The attestation/assertion verification logic is covered by `devicecheck-appattest`'s own test suite; what's untested here is the routing, the Redis single-use semantics, and the idempotent upsert — worth a Testcontainers-based integration suite before this sees production traffic.
- A real schema migration tool (currently just `CREATE TABLE IF NOT EXISTS` on startup, fine for one table, not for an evolving schema).